

[物件導向 Ep. 1] 類別與物件

tags: MCL oop

Declaration

此系列內容強調物件導向的部分，因此不會參雜太多語法介紹。會建議使用 C++ 或是 Java 當作基底語言去學習，C++ 的原因是他有完整的物件導向特性；Java 也有 (甚至有時候還比 C++ 更好理解些)，只是他的開發環境比較囉嗦；Python 本身是全物件導向，但是他有部分類別操作是比較複雜的，所以只會帶一些簡單的例子輔佐。

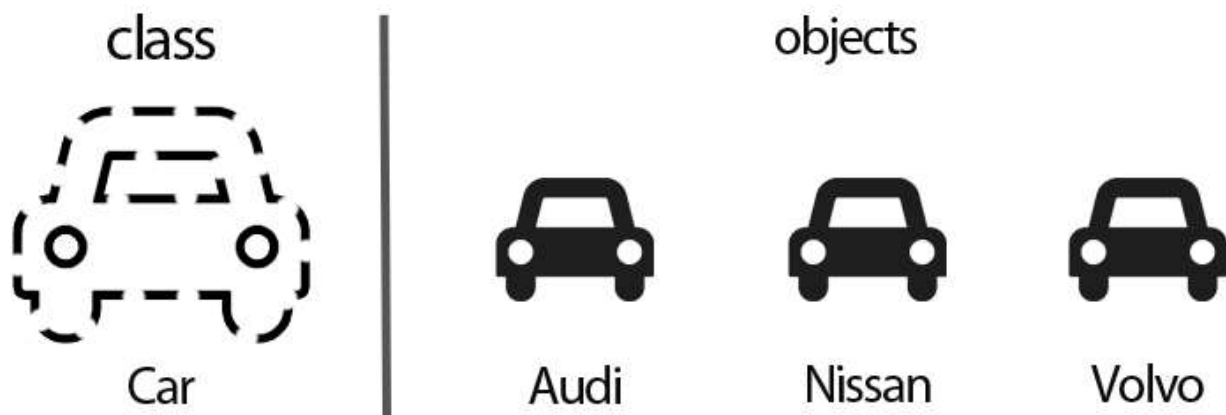
前言

物件導向程式設計 (Object Oriented Programming, OOP) 是一種概念，他傳達著說當我們在操作一團變數與方法的時候，可以朝著

- 把**相關的變數與方法們封裝成一個「東西」**，讓使用者**透過這個東西去操作這些內容物** (可能是變數或是函數方法)。
- 當兩種不同的類型有強烈的歸屬關係，如「水」跟「熱水」，時，「熱水」**可以透過某種方式繼承「水」那邊的內容物** (比如測量公升數方法之類的)，**而不用自己重寫**。
- 且當**多種不種類型有共同的歸屬關係**，如「貓」跟「狗」都是「動物」，「動物」都有叫聲，時，**可以透過某種途徑對每個東西呼叫一樣的函數，但是呈現出不一樣的行為**。一個經典的例子是：貓的叫聲是「喵」，而狗的叫聲是「汪」，但他們都是動物的叫聲。

這些行為可以幫助我們增強程式碼的結構性，並且減少了程式碼的重複性。而實現上面三種行為的基本元素，就是類別 (Class) 與物件 (Object)。

類別與物件的關係



— 🧑 What is Object Oriented Programming (OOP) (<https://javatutorial.net/java-oop>)

上圖是一個有趣的例子，他很簡要地說明了類別跟物件的關係。一言蔽之，

- 類別 (Class) 是**架構定義**，
- 物件 (Object) 是**根據架構宣告出來的變數**。

類別與物件的好處在於可以把其相關的方法包裝在物件 (變數) 當中，而不用特別在外部設計函數，避免還要花功夫理解物件內部到底是怎麼運作的。

案例

舉例來說，在 C++ 當中，

```
1 std::string foo("Alice");  
2 std::cout << foo.length() << std::endl;
```

其中你可以觀察到：

- `std::string` 是一種類別 (型別)，而 `foo` 即是生成出來的物件 (變數)，並且在建立的時候把 "Alice" 字串作為其內部值。
- `length()` 是包裝在 `foo` 物件裡面的一個方法 (函數)，他會負責回傳該字串的長度。
- 你不會看到 `length()` 方法在內部的處理方式，他已經包裝起來了 (封裝 Encapsulation)。

對於 C++ 的字串，你可能比較常看到以下

```
std::string foo = "Alice";
```

的用法，原因是 C++ 會幫你實現複製初始化 (Copy initialization) 的行為，從正統物件初始的寫法，他會是下面的形式：

```
std::string foo("Alice");
```

- 假若使用 `explicit` 關鍵字，copy initialization 就會失效。

另外一個例子，來看 Python 3：

```
1 foo = str('Alice')
2 print(foo.startswith('A')) # string `foo` starts with 'A'
```

同樣地概念，

- `<class 'str'>` 是一種類別 (型態)。惟因為 Python 採動態型別 (宣告變數的時候不需要指派型態)，所以你需要用 `type(foo)` 去查看他的當前型態。
- 而 `foo` 是生成出來的物件 (變數)，並且在建立的時候把 "Alice" 字串作為其內部值。
- `startswith()` 是包裝在 `foo` 物件裡面的一個方法 (函數)，他會檢查該字串是不是由某個字串當起頭。
- 你不會看到 `startswith` 方法內部的處理方式，他已經包裝起來了 (封裝 Encapsulation)。

支援物件導向的語言

在文章開頭處有提到，物件導向本身是一種概念，所以你有機會在不同的語言看到相似的使用法，不同之處可能只在於語法，或者是內部一些細微的差異。

現在大多數的語言都支援物件導向程式設計模式：

- C++
- Java
- Python 2/3
- JavaScript (ES6^)

- PHP
- Ruby
- Kotlin
- etc.

不過也有一些語言是不支援這種模式的，如：

- C
- Golang (Go Language)

他們也有自己的包裝方法：結構 (Structure)，但他是弱化的類別，沒有物件導向的特性。

```
1 // c structure
2 struct student {
3     char name[10];
4     int age;
5 };
6 typedef struct student student;
```

```
1 // golang structure
2 type Student struct {
3     name string
4     age int
5 };
```

類別的用法

從上面的鋪陳中，你依稀會發現一個類別應該涵蓋一些項目：

- 裡面有一些變數和一些方法，我們會把他們稱為成員變數跟成員方法。
- 更細節的來說，你可能會希望有些成員不能讓物件外直接取用 (比如一個字串儲存貓叫聲的變數)。
- 在物件生成的時候，你可以把傳進來的值做些處理，比如把值 "Alice" 儲存到內部的某個變數當中。我們會用一個函數包裝裡面的動作，這個函數有一個特殊的名詞叫建構子 (Constructor)。

了解這些需求後，我們直接看範例：

- C++

```
1  #include <iostream>
2
3  using namespace std;
4
5  class Cat {
6      public:
7          Cat(string animalName) {
8              name = animalName;
9              barking = "meow";
10         }
11         void saySomething() {
12             cout << name << ": " << barking
13                 << endl;
14         }
15
16         private:
17             string name;
18             string barking;
19     };
20
21     int main() {
22         Cat c1("Alice");
23         Cat c2("Bob");
24
25         c1.saySomething();
26         c2.saySomething();
27
28         return 0;
29     }
```

► Java

► Python 3

► JavaScript ES6^

從上面的例子中，你會發現

- 類別名稱是 `Cat`，裡面有兩個變數 `name` `barking` 跟一個方法 `saySomething()`
- 他們都有一個建構子，建構子的參數叫做 `animalName`，過程中他會把他的值指派給成員變數 `name`

物件關鍵字

如果參數名字跟成員變數名字衝突的時候，你可以使用關鍵字去強調你要調用的是成員變數：

- C++: this

```
1  #include <iostream>
2
3  using namespace std;
4
5  class Cat {
6      public:
7          Cat(string name) {
8              this->name = name;
9              this->barking = "meow";
10         }
11         void saySomething() {
12             cout << this->name << ": " << this->barking
13                 << endl;
14         }
15
16         private:
17             string name;
18             string barking;
19     };
20
21     int main() {
22         Cat c1("Alice");
23         Cat c2("Bob");
24
25         c1.saySomething();
26         c2.saySomething();
27
28         return 0;
29     }
```

- this 在 C++ 代表的是指標，調用物件指標之成員的方式是用 ->

- ▶ Java: this
- ▶ Python 3: self
- ▶ JavaScript ES6^: this

公開成員與私有成員

公開與私有不同之處在於：公開成員可以從外部取用，但私有不行。

從上述例子 (C++) 當中，`saySomething()` 是公開成員，所以你可以使用

```
Cat c1("Alice");  
c1.saySomething();
```

去使用他的功能。

但 `name` 跟 `barking` 是私有成員，所以當你做以下任何動作時都會發生錯誤：

```
Cat c1("Alice");  
cout << c1.name; // will raise compile error  
cout << c1.barking; // also will raise compile error
```

這樣的特性並不局限於你的成員是變數還是函數，重點是他是 `public` 還是 `private`。

慣用寫法

習慣來說，我們會把變數跟部份的函數放到私有，而想要讓人使用的函數放到公開去。

但一般程式還是有可能需要存取變數值的需求，但你的選擇不會是把變數放到公開去，這意味著該值可以被使用者任意修改。相對地，針對一個變數，我們會傾向另外開兩個函數空該分別進行取值 (Getter) 跟賦值 (Setter) 行為。

以 `name` 為例，我們可以這麼做：

```
1  class Cat {
2      public:
3          // ...
4
5          string getName() {
6              return this->name;
7          }
8
9          void setName(string name) {
10             if (name.length() > 10) {
11                 cerr << "name.length() should <= 10" << endl;
12             }
13             this->name = name;
14         }
15
16     private:
17         string name;
18 };
```

接著你就可以透過存取 Cat 物件中的 name 成員變數：

```
Cat c1("Alice");
cout << c1.getName() << endl; // Alice
c1.setName("Bob");
cout << c1.getName() << endl; // Bob
```

你會發現當你呼叫 setName() 的時候，他會去檢查你取的名字長度是否超過 10 個字元，如果違反就會輸出錯誤訊息。這便是透過函數控制的好處：你可以加入自己定義的檢查機制，避免你所操作的值會發生不預期的狀況。